

Introduction to Reinforcement Learning

THIRD IN-DEEP WEEK

Ignacio Brevis

`ignacio.brevis@nottingham.ac.uk`

University of Nottingham

13-17 July 2026



University of
Nottingham
UK | CHINA | MALAYSIA



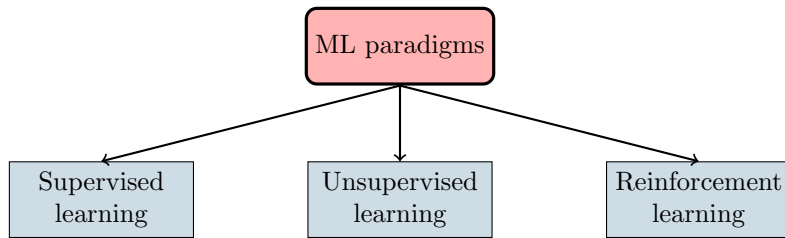
Introduction to reinforcement learning



<https://www.youtube.com/watch?v=spfpBrBjntg>

Introduction to reinforcement learning

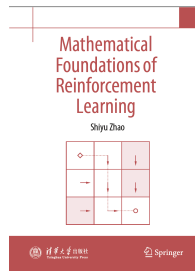
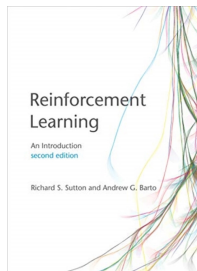
Area		Examples
Robotic Control	&	Robotic grasping and assembly; drone navigation; quadruped locomotion; industrial pick-and-place systems.
Games & decision making		AlphaGo/AlphaZero (Go, Chess); Atari agents (DQN); StarCraft II and Dota 2 self-play systems.
Recommendation & personalization		YouTube video rankings; e-commerce product ordering; personalized news feeds.
Autonomous systems		Self-driving vehicles; warehouse AGV (automated guided vehicle); traffic light coordination; spacecraft docking.
Finance y trading		Portfolio optimization; algorithmic trading; market making; sequential fraud detection.
Healthcare medicine	&	Optimal drug dosing; radiotherapy planning; clinical decision support; robotic rehabilitation systems.



- **Supervised learning:** Model learns from a training set of labeled examples provided by a knowledgeable external supervisor
- **Unsupervised learning:** Model discovers structures or patterns without predefined target values (no labels).
- **Reinforcement learning:** Model learns by interacting with an environment and receiving rewards.

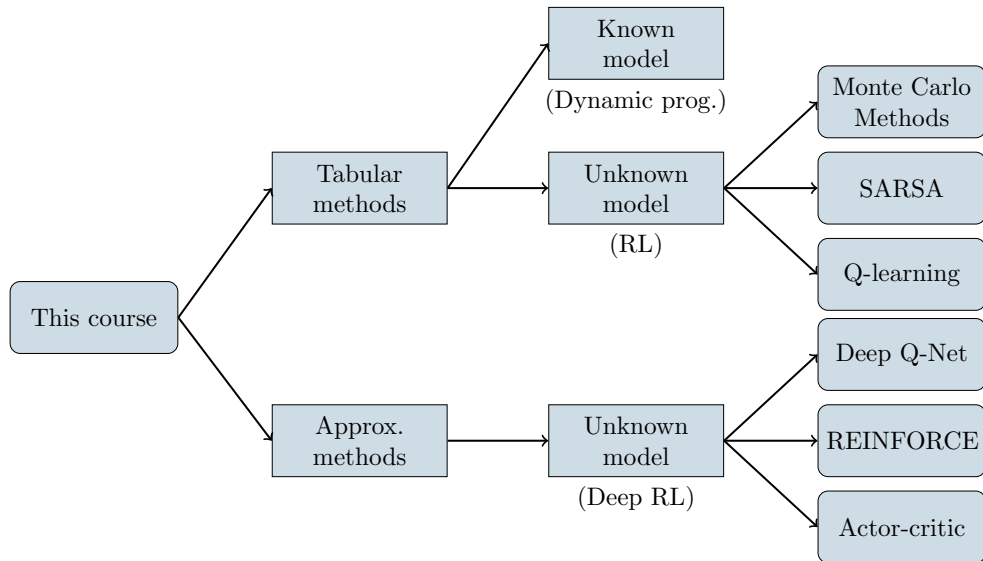
Introduction to reinforcement learning

- Classical RL (tabular case)
 - MDP
 - Bellman equation
 - SARSA
 - Q-learning
- Deep RL (continuous case)
 - Function approximation
 - Deep Q-learning
 - Policy gradient



- R. S. Sutton and A. G. Barto, **Reinforcement Learning: An Introduction**, 2018.
- S. Zhao, **Mathematical Foundations of Reinforcement Learning**, 2025.

Introduction to reinforcement learning



A grid-world example

Grid world example

- The robot agent starts at the top left corner of the grid.
- The agent will explore the grid (*environment*).
- The game objective is for the agent to reach the blue cell (*target*).
- While cells are accessible/allowed.
- Orange cells are forbidden/penalized.

RL: Find an “optimal path” to reach the target cell.

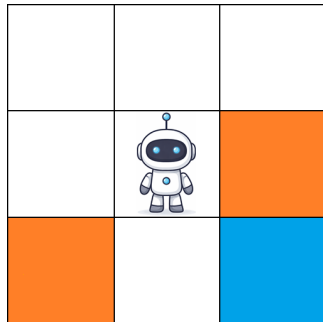


Image adapted from Zhao, Mathematical Foundations of Reinforcement Learning.

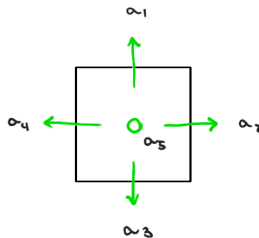
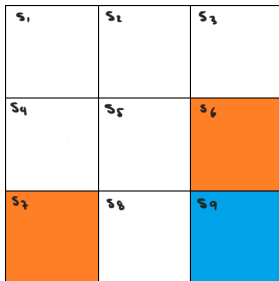
States and actions

The **states** represent the location of the agent in the environment.

$$\mathcal{S} = \{s_1, s_2, s_3, s_4, s_5, s_6, s_7, s_8, s_9\} \quad (\text{state space})$$

The **actions** indicate the possible movements within the environment.

$$\mathcal{A} = \{a_1, a_2, a_3, a_4, a_5\} \quad (\text{action space})$$



Obs: The action space might vary with the states (e.g., $\mathcal{A}(s_1) = \{a_2, a_3, a_5\}$)

State transition

After taking an action, the agent will transition from one state to another; this is known as **state transition**.

For example, the agent is in s_1 y and selects a_2 (going right), the agente should move to s_2

This can be formally denoted by

$$p(s_1|s_1, a_2) = 0$$

$$p(s_2|s_1, a_2) = 1$$

$$p(s_1|s_1, a_2) = 0$$

$$\vdots$$

$$p(s_9|s_1, a_2) = 0$$

s_1	s_2	s_3
s_4	s_5	s_6
s_7	s_8	s_9

Obs: In a stochastic environment, choosing an action can lead to different states.

State transition

After taking an action, the agent will transition from one state to another; this is known as **state transition**.

For example, the agent is in s_1 y and selects a_2 (going right), the agente should move to s_2

This can be formally denoted by

$$p(s_2|s_1, a_2) = 0.2$$

$$p(s_2|s_1, a_2) = 0.8$$

$$p(s_1|s_1, a_2) = 0$$

$$\vdots$$

$$p(s_9|s_1, a_2) = 0$$

s_1	s_2	s_3
s_4	s_5	s_6
s_7	s_8	s_9

Obs: In a stochastic environment, choosing an action can lead to different states.

Policies, denoted by $\pi(a|s)$, define the actions the agent will follow at each state.

They describe the probabilities of taking each action (deterministic case):

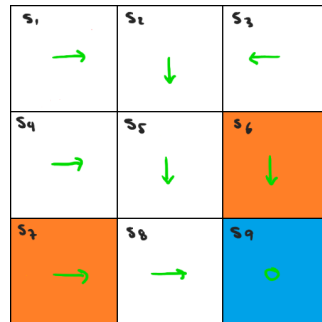
$$\pi(a_1|s_1) = 0$$

$$\pi(a_2|s_1) = 1$$

$$\pi(a_3|s_1) = 0$$

$$\pi(a_4|s_1) = 0$$

$$\pi(a_5|s_1) = 0$$



Policies, denoted by $\pi(a|s)$, define the actions the agent will follow at each state.

They describe the probabilities of taking each action (stochastic case):

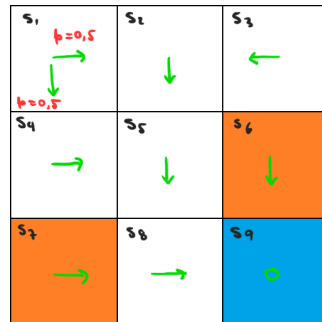
$$\pi(a_1|s_1) = 0$$

$$\pi(a_2|s_1) = 0.5$$

$$\pi(a_3|s_1) = 0.5$$

$$\pi(a_4|s_1) = 0$$

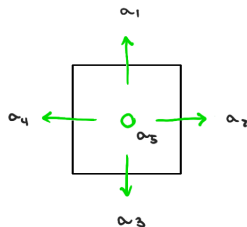
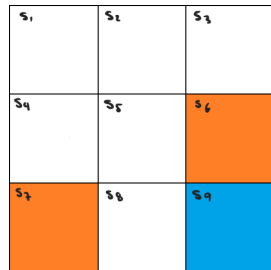
$$\pi(a_5|s_1) = 0$$



Rewards

After taking an action on a given state, the agent obtain a **reward** ($r := r(s, a) \in \mathbb{R}$) as feedback from the environment.

- A “positive” reward will encourage to take such action.
- A “negative” reward will discourage to take that action.



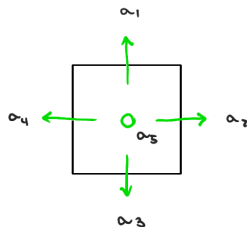
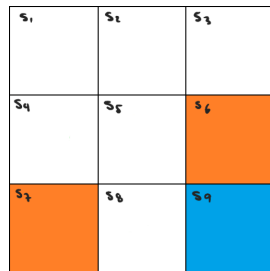
Rewards

After taking an action on a given state, the agent obtain a **reward** ($r := r(s, a) \in \mathbb{R}$) as feedback from the environment.

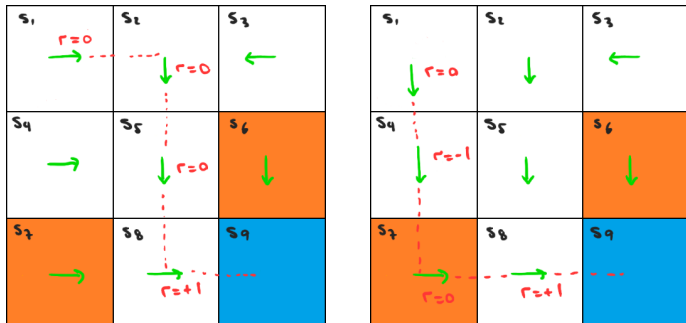
- A “positive” reward will encourage to take such action.
- A “negative” reward will discourage to take that action.

For example, in the grid world example,

- If the agent attempts to go out of the grid, $r = -1$.
- If the agent attempts to enter a forbidden cell, $r = -1$.
- If the agent reaches the target cell, $r = +1$.
- Otherwise, $r = 0$.



Trajectories

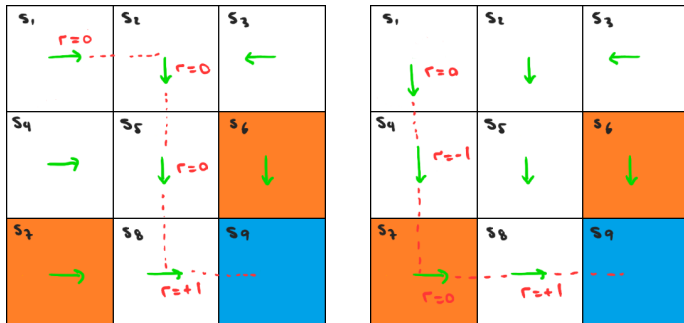


A **trajectory** is a state-action-reward chain (of any length). For examples, given the policy on the left, the agent follows the following chain:

$$s_1 \xrightarrow[a_2]{r=0} s_2 \xrightarrow[a_3]{r=0} s_5 \xrightarrow[a_3]{r=0} s_8 \xrightarrow[a_2]{r=1} s_9$$

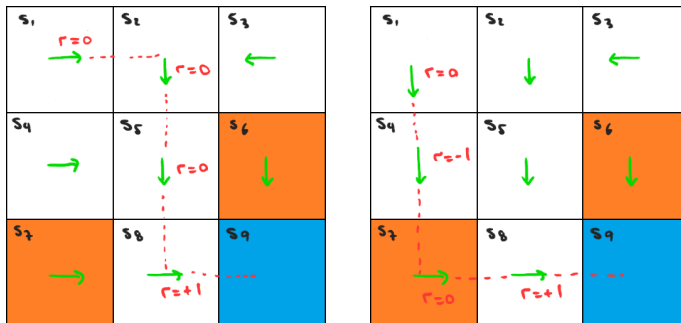
Image adapted from Zhao, Mathematical Foundations of Reinforcement Learning.

Episodes



- An **episode** is a trajectory that ends on a **terminal state**.
- A **terminal state** is a state where the agent-environment interaction stops (or resets).

Image adapted from Zhao, Mathematical Foundations of Reinforcement Learning.



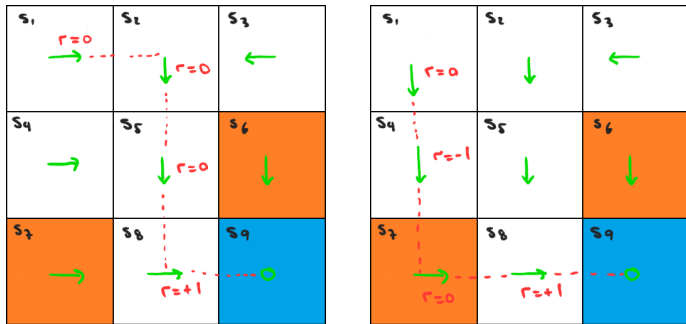
The **return** (total rewards or cumulative rewards) of an episode is the sum of all the rewards obtained during the trajectory:

$$\text{return} = 0 + 0 + 0 + 1 = 1$$

Image adapted from Zhao, Mathematical Foundations of Reinforcement Learning.

- **Episodic tasks:** the agent-environment interaction naturally breaks down into a sequence of separate episodes. Each episode ends in a special terminal state, followed by a reset to a standard starting state.
- **Continuing tasks:** the interaction does not break naturally into identifiable episodes but it keeps going (no terminal state), e.g., on-going process-control task.

Episodic and continuing tasks



return = $0 + 0 + 0 + 1 = 1$ (episodic task)

return = $0 + 0 + 0 + 1 + 1 + 1 + \dots$ (continuing task)

Image adapted from Zhao, Mathematical Foundations of Reinforcement Learning.

To avoid this issue, we introduce the concept of **discount**. Given $\gamma \in (0, 1)$, then:

$$\text{discounted return} = \sum_{i=0}^{\infty} \gamma^i r_{i+1}$$

We can treat episodic and continuing tasks in a unified framework by converting episodic tasks to continuing ones. To do this, we need well define what happens after the agent reaches the terminal state.

- We define terminal states as a special state where the agent stays in this state forever.
- We define terminal state as a normal state and the agent may leave the state and come back again.

A **Markov decision process** (MDP) is a discrete stochastic process defined by:

- Sets:
 - State space $\mathcal{S}$ (state space)
 - Action space $\mathcal{A}(s)$, for any $s \in \mathcal{S}$
 - Reward set $\mathcal{R}(s, a)$, where $s \in \mathcal{S}$ y $a \in \mathcal{A}(s)$
- Model or dynamics
 - State transition probability: $p(s'|s, a)$
 - Reward probability: $p(r|s, a)$
- Policy
 - Probability of choosing action a : $\pi(a|s)$
- Markov property
 - $p(s_{t+1}|s_t, a_t, s_{t-1}, a_{t-1}, \dots, s_0, a_0) = p(s_{t+1}|s_t, a_t)$
 - $p(r_{t+1}|s_t, a_t, s_{t-1}, a_{t-1}, \dots, s_0, a_0) = p(r_{t+1}|s_t, a_t)$

Obs: Notice that, given $s \in \mathcal{S}$ and $a \in \mathcal{A}(s)$,

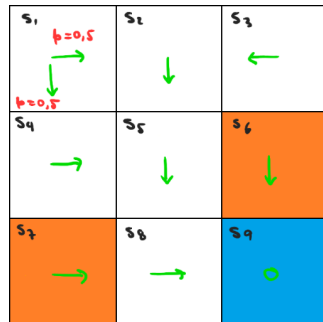
$$\sum_{a \in \mathcal{A}(s)} \pi(a|s) = 1,$$

$$\sum_{s' \in \mathcal{S}} p(s'|s, a) = 1,$$

and

$$\sum_{r \in \mathcal{R}(s, a)} p(r|s, a) = 1,$$

Obs: Once the policy is fixed, the MPD becomes a Markov process.



Let us now consider the states S_t , the actions A_t , and the rewards R_t in terms of random variables. The reward is given by:

$$G_t := \sum_{k=0} \gamma^k R_{t+k+1} = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots$$

Value function (or state-value function)

Given a policy π and $s \in \mathcal{S}$. The **value function** is defined as the expected value of G_t given s :

$$v_{\pi}(s) := \mathbb{E}[G_t | S_t = s]$$

Action-value function

Given a policy π , $s \in \mathcal{S}$ and $a \in \mathcal{A}$. The **action-value function** is defined as the expected value of G_t given s and a :

$$q_{\pi}(s, a) := \mathbb{E}[G_t | S_t = s, A_t = a]$$

Bellman equation

Let's consider $v_{\pi}(s) = \mathbb{E}[G_t | S_t = s]$

Let's consider $v_\pi(s) = \mathbb{E}[G_t | S_t = s]$

$$\begin{aligned} v_\pi(s) &= \mathbb{E}[G_t | S_t = s] \\ &= \mathbb{E}[R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots | S_t = s] \\ &= \mathbb{E}[R_{t+1} + \gamma (R_{t+2} + \gamma R_{t+3} + \dots) | S_t = s] \\ &= \mathbb{E}[R_{t+1} | S_t = s] + \gamma \mathbb{E}[G_{t+1} | S_t = s] \end{aligned}$$

Bellman equation

Let's consider $v_\pi(s) = \mathbb{E}[G_t | S_t = s]$

$$\begin{aligned}v_\pi(s) &= \mathbb{E}[G_t | S_t = s] \\&= \mathbb{E}[R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots | S_t = s] \\&= \mathbb{E}[R_{t+1} + \gamma (R_{t+2} + \gamma R_{t+3} + \dots) | S_t = s] \\&= \mathbb{E}[R_{t+1} | S_t = s] + \gamma \mathbb{E}[G_{t+1} | S_t = s]\end{aligned}$$

$$\mathbb{E}[R_{t+1} | S_t = s]$$

$\downarrow$

$$\sum_{a \in \mathcal{A}} \pi(a|s) \sum_{r \in \mathcal{R}} p(r|s, a) r$$

$$\mathbb{E}[G_{t+1} | S_t = s]$$

$\downarrow$

$$\sum_{a \in \mathcal{A}} \pi(a|s) \sum_{s' \in \mathcal{S}} p(s'|s, a) v_\pi(s')$$

The **Bellman equation** is given by:

$$\begin{aligned} v_{\pi}(s) &= \sum_{a \in \mathcal{A}} \pi(a|s) \sum_{r \in \mathcal{R}} p(r|s, a)r + \gamma \sum_{a \in \mathcal{A}} \pi(a|s) \sum_{s' \in \mathcal{S}} p(s'|s, a)v_{\pi}(s'), \\ &= \sum_{a \in \mathcal{A}} \pi(a|s) \left[\sum_{r \in \mathcal{R}} p(r|s, a)r + \gamma \sum_{s' \in \mathcal{S}} p(s'|s, a)v_{\pi}(s') \right], \quad \forall s \in \mathcal{S} \end{aligned}$$

- Notice that $v_{\pi}(s)$ y $v_{\pi}(s')$ are the unknowns (assuming we know the model).
- Given a policy $\pi := \pi(a|s)$, the value function give us a **value for the policy**.
- $p(r|s, a)$ and $p(s'|s, a)$ define the dynamic of the system.
- To solve this equation, we can generate a linear system of $n = \dim \mathcal{S}$ equations.

Given

$$r_{\pi}(s) := \sum_{a \in \mathcal{A}} \pi(a|s) \sum_{r \in \mathcal{R}} p(r|s, a) r, \quad \text{and} \quad p_{\pi}(s'|s) := \sum_{a \in \mathcal{A}} \pi(a|s) p(s'|s, a)$$

- $r_{\pi}(s)$ denotes the expected value of the (immediate) reward under the policy π .
- $p_{\pi}(s'|s)$ represents the transition probability from s to s' under the policy π .

Then, for each s_i , $i = 1, \dots, n$ ($n = |\mathcal{S}|$), we can rewrite the Bellman equation as follows:

$$\boxed{v_{\pi}(s_i) = r_{\pi}(s_i) + \gamma \sum_{s_j \in \mathcal{S}} p_{\pi}(s_j|s_i) v_{\pi}(s_j)} \quad (\text{bootstrapping})$$

Bellman equation

Defining:

$$v_\pi = \begin{bmatrix} v_\pi(s_1) \\ v_\pi(s_2) \\ \vdots \\ v_\pi(s_n) \end{bmatrix}, \quad r_\pi = \begin{bmatrix} r_\pi(s_1) \\ r_\pi(s_2) \\ \vdots \\ r_\pi(s_n) \end{bmatrix}, \quad P_\pi = \begin{bmatrix} p_\pi(s_1|s_1) & p_\pi(s_2|s_1) & \cdots & p_\pi(s_n|s_1) \\ p_\pi(s_1|s_2) & p_\pi(s_2|s_2) & \cdots & p_\pi(s_n|s_2) \\ \vdots & & \ddots & \vdots \\ p_\pi(s_1|s_n) & p_\pi(s_2|s_n) & \cdots & p_\pi(s_n|s_n) \end{bmatrix}$$

where $v_\pi \in \mathbb{R}^n$, $r_\pi \in \mathbb{R}^n$ and $P_\pi \in \mathbb{R}^{n \times n}$.

Bellman equation can be written in a matrix form:

$$\boxed{v_\pi = r_\pi + \gamma P_\pi v_\pi} \quad \Rightarrow \quad \boxed{(I - \gamma P_\pi)v_\pi = r_\pi}$$

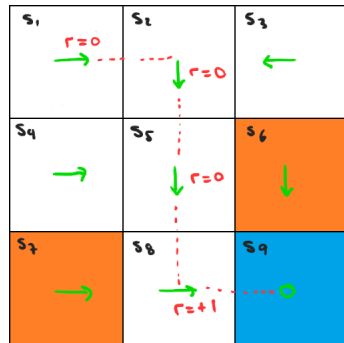
Obs: Since P_π is an stochastic matrix (rows are nonnegative and sum to 1) and $\gamma < 1$, **the system has a unique solution** (proof using the Gershgorin circle theorem).

Bellman equation

Example:

In https://github.com/ibrevis/in-deep_week_intro_RL,

- go to `/simple_grid_world/`
- then `01_Bellman_equation.ipynb`
- compute the value function for the given policy
- what happens if $s_5 \rightarrow s_4$ and $s_4 \rightarrow s_7$ instead?



In terms of the action values (q -values):

$$v_{\pi}(s) = \mathbb{E}[G_t | S_t = s] = \sum_{a \in \mathcal{A}} \pi(a|s) \mathbb{E}[G_t | S_t = s, A_t = a] = \sum_{a \in \mathcal{A}} \pi(a|s) q(s, a)$$

In addition, from $q_{\pi}(s, a) = \mathbb{E}[G_t | S_t = s, A_t = a]$, we can obtain:

$$q_{\pi}(s, a) = \sum_{r \in \mathcal{R}} p(r|s, a) r + \gamma \sum_{s' \in \mathcal{S}} p(s'|s, a) v_{\pi}(s')$$

and

$$q_{\pi}(s, a) = \sum_{r \in \mathcal{R}} p(r|s, a) r + \gamma \sum_{s' \in \mathcal{S}} p(s'|s, a) \sum_{a' \in \mathcal{A}(s')} \pi(a'|s') q_{\pi}(s', a')$$

Optimal policy

A policy π^* is **optimal** if the value function follows that:

$$v_{\pi^*}(s) \geq v_{\pi}(s), \quad \forall s \in \mathcal{S}$$

for any other policy π .

Obs: $v_{\pi^*}(s)$ is the **optimal value** for the state s .

We are interested on :

- Existence?
- Uniqueness?
- Algorithms?

Bellman optimality equation (BOE)

$$v(s) = \max_{\pi(s)} \sum_{a \in \mathcal{A}} \pi(a|s) q(s, a),$$

where

$$q(s, a) = \sum_{r \in \mathcal{R}} p(r|s, a) r + \gamma \sum_{s' \in \mathcal{S}} p(s'|s, a) v(s')$$

- The BOE “solves the Bellman equation”, but instead taking a given policy, it search for the policy that maximizes $v(s)$.
- The BOE allow us to find **optimal policies** and **optimal value functions**.

Bellman optimality equation

Given an n -state environment n such that $\mathcal{S} = \{s_1, s_2, \dots, s_n\}$, then the BOE can be written as :

$$\begin{bmatrix} v(s_1) \\ v(s_2) \\ \vdots \\ v(s_n) \end{bmatrix} = \max_{\pi \in \Pi} \left(\begin{bmatrix} r_\pi(s_1) \\ r_\pi(s_2) \\ \vdots \\ r_\pi(s_n) \end{bmatrix} + \gamma \begin{bmatrix} p_\pi(s_1|s_1) & p_\pi(s_2|s_1) & \cdots & p_\pi(s_n|s_1) \\ p_\pi(s_1|s_2) & p_\pi(s_2|s_2) & \cdots & p_\pi(s_n|s_2) \\ \vdots & \vdots & \ddots & \vdots \\ p_\pi(s_1|s_n) & p_\pi(s_2|s_n) & \cdots & p_\pi(s_n|s_n) \end{bmatrix} \begin{bmatrix} v(s_1) \\ v(s_2) \\ \vdots \\ v(s_n) \end{bmatrix} \right),$$

$$\text{where } r_\pi(s) := \sum_{a \in \mathcal{A}} \pi(a|s) \sum_{r \in \mathcal{R}} p(r|s, a)r \quad \text{and} \quad p_\pi(s'|s) := \sum_{a \in \mathcal{A}} \pi(a|s)p(s'|s, a)$$

BOE matrix form

$$v = \max_{\pi \in \Pi} (r_\pi + \gamma P_\pi v)$$

Resolución de la BOE

Solving the BOE leads to find v^* and an optimal policy π^* such that:

$$v^* = \max_{\pi \in \Pi} (r_{\pi} + \gamma P_{\pi} v^*)$$

Obs: $f(v) := \max_{\pi \in \Pi} (r_{\pi} + \gamma P_{\pi} v)$ is a contraction mapping.

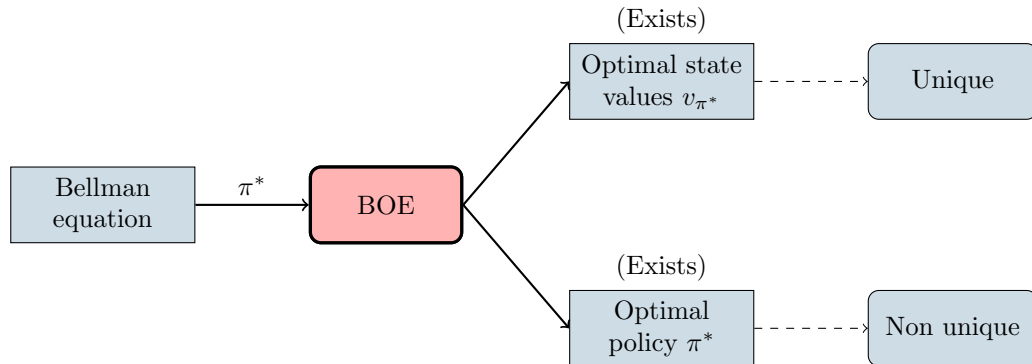
The Bellman optimality equation can be solve with the Banach fixed-point theorem (contraction mapping thm)!

Then, it exists a unique v^* that solves the BOE. In addition, we can compute v^* iteratively

$$v_{k+1} = f(v_k) = \max_{\pi \in \Pi} (r_{\pi} + \gamma P_{\pi} v_k),$$

where $v_k \rightarrow v^*$ ($k \rightarrow \infty$). Notice that, $\pi^* \in \arg \max_{\pi \in \Pi} (r_{\pi} + \gamma P_{\pi} v^*)$ might not be unique.

Bellman optimality equation



Obs: We can always find a **deterministic** optimal policy (following a **greedy** approach)

Optimal policy

A policy π^* is **optimal** if the value function follows that:

$$v_{\pi^*}(s) \geq v_{\pi}(s), \quad \forall s \in \mathcal{S}$$

for any other policy π .

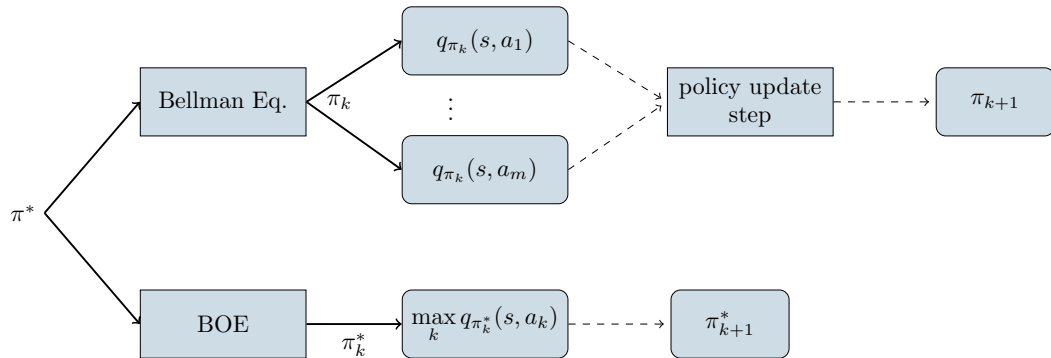
Obs: $v_{\pi^*}(s)$ is the **optimal value** for the state s .

We are interested on :

- Existence? ✓
- Uniqueness? (value function ✓ ; policy ✗)
- Algorithms?

RL Algorithms

Main idea:



How do we compute q ?

- **Known model:** Knowing the probability distribution, we can compute

$$\mathbb{E}[X] = \sum_x p(X = x)x$$

- **Unknown model:** When the probability distribution (i.e., the model) of X is unknown. We can estimate the expected value from samples $\{x_1, x_2, \dots, x_n\}$ of X .

$$\mathbb{E}[X] \approx \bar{x} := \frac{1}{n} \sum_{j=1}^n x_j$$

- **Monte Carlo methods do not require the probability distribution.** This is important here since:

$$v_{\pi}(s) = \mathbb{E}[G_t | S_t = s]$$

$$q_{\pi}(s, a) = \mathbb{E}[G_t | S_t = s, A_t = a]$$

- **Known model:**

$$q_{\pi_k}(s, a) = \sum_r p(r|s, a)r + \gamma \sum_{s'} p(s'|s, a)v_{\pi_k}(s')$$

- **Unknown model:**

$$q_{\pi_k} = \mathbb{E}[G_t | S_t = s, A_t = a]$$

where $G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots$

Estimación de MC

Suppose that there are n episodes (starting at s and selecting a). The return of each episode is $g_{\pi_k}^{(i)}(s, a)$. Then, $q_{\pi_k}(s, a)$ can be approximated as:

$$q_{\pi_k}(s, a) = \mathbb{E}[G_t | S_t = s, A_t = a] \approx \frac{1}{n} \sum_{i=1}^n g_{\pi_k}^{(i)}(s, a)$$

- **Pros:**
 - **model-free and easy to implement**
 - **unbiased:** *iid.* samples
 - **no bootstrapping:** it can focus on some states (no need to sweeping over all states)
- **Cons:**
 - **full episodes:** works better on episodic tasks
 - **optimal policies:** needs a policy update step for computing them
 - **high variance:** sum of many random transition and rewards
 - **exploration issue:** in deterministic policies you only ever see one action (partially solved with MC exploring starts)
 - **incomplete theory:** convergence of MC-ES to the optimal fixed point has never been proved (open question in RL) [Sutton & Barto, Section 5.3]

Stochastic approximation refers to a broad class of stochastic iterative algorithms solving root finding or optimization problems.

Robbins-Monro (RM) algorithm (1951), allow us to solve problems like:

$$g(w) = 0,$$

where **g is unknown** and we only have access to **noisy observations** $\tilde{g}(w, \eta) := g(w) + \eta$. Then, the RM algorithm follows:

$$w_{k+1} = w_k - \alpha_k \tilde{g}(w_k, \eta_k)$$

H. Robbins and S. Monro, A Stochastic Approximation Method, *Ann. Math. Stat. or Ann. Math. Statist.*, 1951.

A STOCHASTIC APPROXIMATION METHOD¹

BY HERBERT ROBBINS AND SUTTON MONRO

University of North Carolina

1. Summary. Let $M(x)$ denote the expected value at level x of the response to a certain experiment. $M(x)$ is assumed to be a monotone function of x but is unknown to the experimenter, and it is desired to find the solution $x = \theta$ of the equation $M(x) = \alpha$, where α is a given constant. We give a method for making successive experiments at levels $x_1, x_2, \dots$ in such a way that x_n will tend to θ in probability.

2. Introduction. Let $M(x)$ be a given function and α a given constant such that the equation

$$(1) \quad M(x) = \alpha$$

has a unique root $x = \theta$. There are many methods for determining the value of θ by successive approximation. With any such method we begin by choosing one or more values $x_1, \dots, x_r$ more or less arbitrarily, and then successively obtain new values x_n as certain functions of the previously obtained $x_1, \dots, x_{n-1}$, the values $M(x_1), \dots, M(x_{n-1})$, and possibly those of the derivatives $M'(x_1), \dots, M'(x_{n-1})$, etc. If

$$(2) \quad \lim_{n \rightarrow \infty} x_n = \theta,$$

Repaso: Robbins-Monro algorithm

Expected value application. Given

- $g(w) = w - \mathbb{E}[X]$
- $\tilde{g}(w, \eta) = w - x$ (where x is a *iid.* sample of X)

We search for w^* such that $g(w^*) = 0$. Notemos que:

$$\begin{aligned}\tilde{g}(w, \eta) &= w - x \\ &= (w - \mathbb{E}[X]) + (\mathbb{E}[X] - x) \\ &= g(w) + \eta\end{aligned}$$

Then, given a sequence of *iid.* samples $\{x_i\}_{i=1}^n$, we can estimate the $\mathbb{E}[X]$ by considering the Robbins-Monro algorithm:

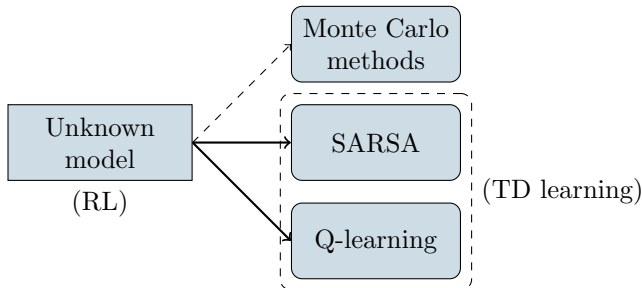
$$w_{k+1} = w_k - \alpha_k(w_k - x_k)$$

Obs: if $\alpha_k = 1/k$ we recover the Monte Carlo method.

Temporal-difference (TD) methods

Temporal-difference (TD) learning methods combine ideas from Monte Carlo methods and dynamic programming (DP).

- **MC**: Does **not require a model** and learns from experiences
- **DP**: Require a model and uses **bootstrapping**



Temporal-difference (TD) methods

Bellman equation (state-value function case)

$$v_{\pi}(s) = \mathbb{E}[R_{t+1} + \gamma G_{t+1} | S_t = s], \quad s \in \mathcal{S}$$

Bellman expectation equation

$$v_{\pi}(s) = \mathbb{E}[R_{t+1} + \gamma v_{\pi}(S_{t+1}) | S_t = s], \quad s \in \mathcal{S}$$

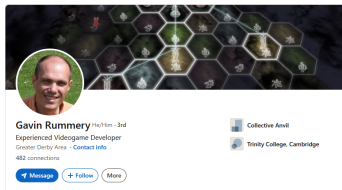
Robbins-Monro para Bellman expectation equation

$$g(v(s)) := v(s) - \mathbb{E}[R_{t+1} + \gamma v_{\pi}(S_{t+1}) | S_t = s]$$

$$\tilde{g}(v(s)) := v(s) - (r_{t+1} + \gamma v_{\pi}(s_{t+1}))$$

$$v_{k+1}(s_t) := v_k(s_t) - \alpha_k \left[v_k(s_t) - (r_{t+1} + \gamma v_{\pi}(s_{t+1})) \right]$$

TD methods: Learning action values (SARSA)



ON-LINE Q-LEARNING USING CONNECTIONIST SYSTEMS

G. A. Rummery & M. Niranjan

CUED/F-INFENG/TR 166

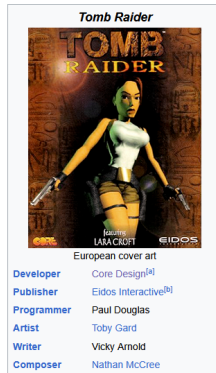
September 1994

Cambridge University Engineering Department
Trumpington Street
Cambridge CB2 1PZ
England

Email: gar@eng.cam.ac.uk, niranjan@eng.cam.ac.uk

Tomb Raider is a 1996 [action-adventure video game](#) developed by [Core Design](#) and published by [Eidos Interactive](#) as the debut entry in the *Tomb Raider* media franchise. It was first released on the [Sega Saturn](#), followed shortly by versions for [MS-DOS](#) and the [PlayStation](#). Later releases came for [Mac OS](#) (1999), [Pocket PC](#) (2002), [N-Gage](#) (2003), [iOS](#) (2013) and [Android](#) (2015). The game follows archaeologist-adventurer [Lara Croft](#), who is hired by businesswoman Jacqueline Natla to find an artefact called the Scion of [Atlantis](#). Gameplay features Lara navigating levels split into multiple areas and room complexes while fighting enemies and solving puzzles to progress.

Tomb Raider was praised for its innovative 3D graphics, controls, and gameplay. The game went on to win several industry awards and is considered to be one of the [greatest video games ever made](#). It is also one of the best-selling games for the PlayStation, with seven million units sold worldwide, and it remained the best-selling title in the *Tomb Raider* franchise until the [2013 reboot](#). Lara Croft herself became a cultural icon, rising to prominence as one of gaming's most recognisable characters. Following the game's success, numerous sequels were released, beginning with *Tomb Raider II* in 1997. Two [remakes](#) have been developed; *Tomb Raider: Anniversary* (2007), and the upcoming *Tomb Raider: Legacy of Atlantis* (2026). A [remastered version](#) of *Tomb Raider*, alongside *Unfinished Business*, was released as part of *Tomb Raider I–III Remastered* in 2024.



TD methods: Learning action values (SARSA)

Given a policy π , we can estimate the action values $q_\pi(s, a)$. Suppose that we have some experience samples generated by π :

$$(s_0, a_0, r_1, \dots, s_t, a_t, r_{t+1}, s_{t+1}, a_{t+1}, \dots, r_T, s_T, a_T)$$

We can use the following algorithm to estimate the action value (known as SARSA):

$$\begin{cases} q_{t+1}(s_t, a_t) = q_t(s_t, a_t) + \alpha_t(s_t, a_t) [r_{t+1} + \gamma q_t(s_{t+1}, a_{t+1}) - q_t(s_t, a_t)], \\ q_{t+1}(s, a) = q_t(s, a), \quad \forall (s, a) \neq (s_t, a_t) \end{cases}$$

where $t = 0, 1, 2, \dots$

- $q_t(s_t, a_t)$ converges to $q_\pi(s_t, a_t)$.
- $\alpha_t(s_t, a_t)$ is the learning rate (depending on s_t and a_t).
- $r_{t+1} + \gamma q_t(s_{t+1}, a_{t+1})$ is known as TD target.
- it needs a policy update step for computing optimal policies ($\rightarrow \epsilon$ -greedy).

TD methods: Learning action values (SARSA)

- **Pros:**
 - **model-free**
 - **online and incremental:** update after every step (not at the end of the episode as MC)
 - **low variance:** few random variables at each step ($S_t, A_t, R_{t+1}, S_{t+1}, A_{t+1}$)
 - **accounts for exploration:** ε -greedy will randomly select an action “ $\varepsilon\%$ of the times”
- **Cons:**
 - **bootstrapping:** introduce bias
 - **optimal policies:** needs a policy update step for computing them
 - **ε -optimal:** converges to π_ε^* , not π^* (unless $\varepsilon \rightarrow 0$)

TD methods: Learning optimal action values (Q-learning)

- Q-learning was developed by Christopher Watkins in 1989 as part of his doctoral thesis at King's College.
- Q-learning is one of the most well known algorithms in reinforcement learning.
- SARSA estimates the value function for a given policy, so it needs a policy update step to find optimal policies.
- Q-learning seeks for optimal actions and policies!



“I connected RL to Markov decision processes and invented Q-learning in the late 1980s in my unpublished PhD thesis”

Image from <https://www.cs.rhul.ac.uk/~chrisw/>

TD methods: Learning optimal action values (Q-learning)

Instead of solving the Bellman expectation equation

$$q_{\pi}(s, a) = \mathbb{E}[R_{t+1} + \gamma q_{\pi}(S_{t+1}, A_{t+1}) | S_t = s, A_t = a]$$

It directly solves the BOE in terms of the action value function:

$$q_{\pi}(s, a) = \mathbb{E}[R_{t+1} + \gamma \max_{a'} q_{\pi}(S_{t+1}, a') | S_t = s, A_t = a]$$

The Q-learning algorithm is given by:

$$\begin{cases} q_{t+1}(s_t, a_t) = q_t(s_t, a_t) + \alpha_t(s_t, a_t) \left[r_{t+1} + \gamma \max_{a'} q_t(s_{t+1}, a') - q_t(s_t, a_t) \right], \\ q_{t+1}(s, a) = q_t(s, a), \quad \forall (s, a) \neq (s_t, a_t) \end{cases}$$

TD methods: Learning optimal action values (Q-learning)

- **Pros:**

- **model-free**
- **online and incremental:** update after every step (not at the end of the episode as MC)
- **lower variance:** fewer random variables at each step ($S_t, A_t, R_{t+1}, S_{t+1}$)
- **optimal policies:** No policy update step (directly learns q^*)

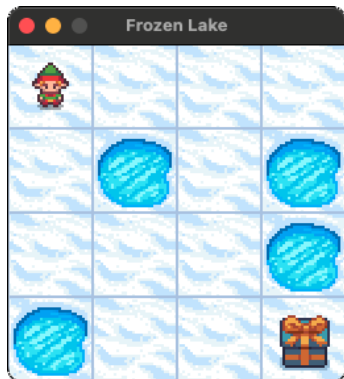
- **Cons:**

- **bootstrapping:** introduce bias
- **exploration issue:** always selecting $\max_a q$ generates problems in noisy environments.
- **convergence:** it can diverge on more general settings (e.g., when combined with neural networks)

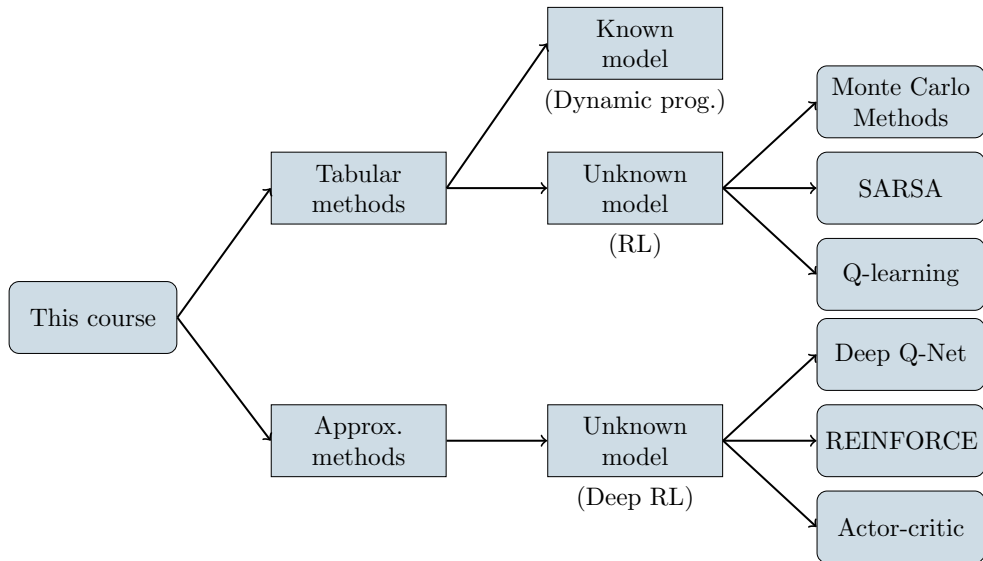
TD methods: Learning optimal action values

In https://github.com/ibrevis/in-deep_week_intro_RL,

- go to `/frozen_lake_example/`
- open `01_FrozenLake_intro.ipynb` for a guided introduction to the environment
- Go `03_SARSA.ipynb` and complete the `q_table` update in the Training section.
- Go `04_Q-learning.ipynb` and complete the `q_table` update in the Training section.
- Train the policies and check your results with `05_run_policy_4x4.py`.
- Explore the case when `map_name="8x8"` and `is_slippery=True` in the Environment section. (check your results with `05_run_policy_8x8.py`)



Deep reinforcement learning



Tabular methods work well for

- Problems like grid world (academic purpose)
- Board games with “reasonable” number of states
- MDP with hundreds or (a few) thousand states
- Problems with finite number of states and actions

Approximate solution methods become relevant when:

- Continuous state spaces (robot joints, velocities, sensor measurements)
- High dimensional inputs (Atari games with 84×84 pixels)
- Complex simulations (self driving cars)
- Large discrete spaces (board game Go, with $\sim 10^{170}$ states) [$\sim 10^{80}$ atoms in the observable universe]

Objective: Since the spaces are too large for the tabular case, we set a **parametric function** $\hat{q}(\cdot, \cdot; w)$ to approximate q_π :

	a_1	a_2	$\dots$	a_m	
s_1	$q(s_1, a_1)$	$q(s_1, a_2)$	$\dots$	$q(s_1, a_m)$	$\leftarrow \hat{q}(s, a; w)$
s_2	$q(s_2, a_1)$	$q(s_2, a_2)$	$\dots$	$q(s_2, a_m)$	
$\vdots$	$\vdots$		$\ddots$	$\vdots$	
s_n	$q(s_n, a_1)$	$q(s_n, a_2)$	$\dots$	$q(s_n, a_m)$	

- Polynomials
- Fourier basis
- Radial basis functions (RBFs)
- Neural networks!

Obs: This idea can also be applied for the value function v_π and the policy itself π .

- First successfully attempt employing Deep NNs in RL.
- Presented at NIPS Deep Learning Workshop, 2013.
- Later published in Nature under the name “Human-level control through deep reinforcement learning” 2015.
- Learned to play Atari games using CNN (achieving professional human-level performance in many cases)



Figure 1: Screen shots from five Atari 2600 Games: (Left-to-right) Pong, Breakout, Space Invaders, Seaquest, Beam Rider

Playing Atari with Deep Reinforcement Learning

Volodymyr Mnih Koray Kavukcuoglu David Silver Alex Graves Ioannis Antonoglou

Daan Wierstra Martin Riedmiller

DeepMind Technologies

{vlad,koray,david,alex.graves,ioannis,daan,martin.riedmiller} @ deepmind.com

Abstract

We present the first deep learning model to successfully learn control policies directly from high-dimensional sensory input using reinforcement learning. The model is a convolutional neural network, trained with a variant of Q-learning, whose input is raw pixels and whose output is a value function estimating future rewards. We apply our method to seven Atari 2600 games from the Arcade Learning Environment, with no adjustment of the architecture or learning algorithm. We find that it outperforms all previous approaches on six of the games and surpasses a human expert on three of them.

Image from Playing Atari with Deep Reinforcement Learning, 2013.

Recalling the Bellman optimality equation (BOE):

$$q(s, a) = \mathbb{E}[R_{t+1} + \gamma \max_{a'} q(S_{t+1}, a') | S_t = s, A_t = a]$$

Deep Q-learning seeks to minimize the following functional:

$$J(w) = \frac{1}{2} \mathbb{E} \left[\left(R + \gamma \max_{a \in \mathcal{A}(S')} \hat{q}(S', a; w) - \hat{q}(S, A; w) \right)^2 \right],$$

where (S, A, R, S') are random variables.

- The objective function can be viewed as the squared Bellman optimality error.
- Since we cannot compute the expected value, the r.v.s S , A , R , and S' are replaced by samples generated by a (behaviour) policy.
- Deep Q-learning will approximate $q_{\pi^*}(s, a)$ (through $\hat{q}(s, a; w)$) and find the (target) policy π^* .

To minimize J , we need to compute $\nabla_w J(w)$, which is not an easy task,

$$J(w) = \frac{1}{2} \mathbb{E} \left[\left(R + \gamma \max_{a \in \mathcal{A}(S')} \hat{q}(S', a; w) - \hat{q}(S, A; w) \right)^2 \right],$$

Deep Q-learning proposes to use two neural networks:

$$J(w) = \frac{1}{2} \mathbb{E} \left[\left(R + \gamma \max_{a \in \mathcal{A}(S')} \hat{q}(S', a; \eta) - \hat{q}(S, A; w) \right)^2 \right]$$

such that the gradient can be easily computed for a fixed η :

$$\nabla_w J(w) = \mathbb{E} \left[\left(R + \gamma \max_{a \in \mathcal{A}(S')} \hat{q}(S', a; \eta) - \hat{q}(S, A; w) \right) \nabla_w \hat{q}(S, A; w) \right]$$

Deep Q-learning computes **optimal actions** through the TD algorithm:

$$w_{t+1} = w_t + \alpha_t \left(r_{t+1} + \gamma \max_{s' \in \mathcal{A}(s_{t+1})} \hat{q}(s_{t+1}, a'; \eta) - \hat{q}(s_t, a_t; w_t) \right) \nabla_w \hat{q}(s_t, a_t; w_t)$$

where $t = 0, 1, 2, \dots$

- We only update η after a few steps. Then, $\eta = w$
- For each t , we take $\{(s_t, a_t, r_{t+1}, s_{t+1})\}$ from a replay buffer, i.e., the samples are shuffled before employed to break correlations (**experience replay**)
- The neural network inputs the state s and outputs $[\hat{q}(s, a_1), \hat{q}(s, a_2), \dots, \hat{q}(s, a_m)]^\top$

$$s \rightarrow \boxed{nn(\cdot; w)} \rightarrow \begin{bmatrix} \hat{q}(s, a_1) \\ \hat{q}(s, a_2) \\ \vdots \\ \hat{q}(s, a_m) \end{bmatrix}$$

Cautionary tale: Combining these three elements can cause instability in the algorithms.

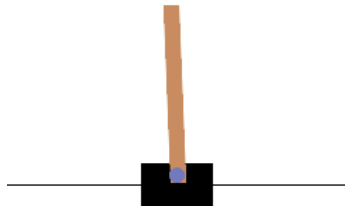
- **Function approximation:** Instead of the tabular case (linear or non linear).
- **Bootstrapping:** Such as temporal difference (TD) learning algorithms.
- **Off-policy training:** when the samples are generated by a policy (behavior policy) different from the target policy.

Example: Baird's counterexample [Sutton & Barto, Section 11.2].

Obs: Many open problems in RL fall into this area.

In https://github.com/ibrevis/in-deep_week_intro_RL,

- go to `/cart_pole_example/`
- open `01_CartPole_intro.ipynb` for a guided introduction to the environment
- Go `03_DQN_CartPole.ipynb` and tune the hyperparameters to training the policy.
- Check your results with `03_run_dqn_cartpole_policy.py`.
- Explore environments "Acrobot-v1" and "LunarLander-v3" and train policies with DQN.



CartPole

The states are given by 4 entries corresponding to the following positions and velocities:

Index	Observation	Minimum	Maximum
0	Cart position	-4.8	4.8
1	Cart velocity	$-\infty$	∞
2	Pole angle	-0.418 rad (-24°)	0.418 rad (24°)
3	Pole angular velocity	$-\infty$	∞

The cart position must remain within $(-2.4, 2.4)$, and the pole angle must remain within approximately $(-0.2095, 0.2095)$ radians, (approximately $\pm 12^\circ$).

Two actions:

- 0: Push cart to the left
- 1: Push cart to the right

By default, the environment provides a reward of +1 at every time step, including the termination step. The default reward threshold for **CartPole-v1** is 500, and the policy is considered successful with an average reward of 475.0 (or larger) over 100 consecutive trials.

Acrobot

The system consists of two links connected in a chain. The first joint is fixed, while torque can be applied to the joint between the two links. The states are given by 6 entries:

Index	Observation	Minimum	Maximum
0	$\cos(\theta_1)$	-1	1
1	$\sin(\theta_1)$	-1	1
2	$\cos(\theta_2)$	-1	1
3	$\sin(\theta_2)$	-1	1
4	Angular velocity $\dot{\theta}_1$	-4π	4π
5	Angular velocity $\dot{\theta}_2$	-9π	9π

Here, θ_1 is the angle of the first link, with $\theta_1 = 0$ corresponding to the link pointing directly downwards. The angle θ_2 is measured relative to the first link.

Three actions:

- 0: Apply a torque of -1 to the actuated joint
- 1: Apply zero torque to the actuated joint
- 2: Apply a torque of $+1$ to the actuated joint

The objective is to swing the free end of the two-link system above the target height. At every step before reaching the goal, the environment provides a reward of -1 . Reaching the target height terminates the episode and provides a reward of 0 . The default reward threshold for **Acrobot-v1** is -100 , meaning that successful policies should reach the target in approximately 100 steps or fewer on average.

LunarLander

The state is given by 8 entries describing the position, velocity, orientation, and ground contact of the lander:

Index	Observation	Minimum	Maximum
0	Horizontal position x	-2.5	2.5
1	Vertical position y	-2.5	2.5
2	Horizontal velocity v_x	-10	10
3	Vertical velocity v_y	-10	10
4	Lander angle θ	-2π	2π
5	Angular velocity $\dot{\theta}$	-10	10
6	Left leg in contact with ground	0	1
7	Right leg in contact with ground	0	1

The landing pad is centred at $(0, 0)$. The last two entries are Boolean variables indicating whether the corresponding leg is touching the ground.

Four discrete actions:

- 0: Do nothing
- 1: Fire left engine
- 2: Fire main eng.
- 3: Fire right eng.

At every step, the reward increases when the lander moves closer to the landing pad, moves more slowly, and remains approximately horizontal. Each leg in the ground contributes $+10$ reward points. Firing an orientation engine incurs a penalty of -0.03 per step, while firing the main engine incurs a penalty of -0.3 per step. A safe landing provides an additional reward of $+100$, whereas crashing produces an additional reward of -100 .

The episode terminates when the lander crashes, moves outside the viewport, or comes to rest. The environment is considered solved when the average episode reward is at least 200.

Policy gradient methods

In the previous examples, we represented the policies as “tables”, where we could change the policy in a state independently of the other states. Now, we describe the **policy** through a **parametric function** $\pi(a|s; \theta)$, with $\theta \in \mathbb{R}^d$

	a_1	a_2		a_m	
s_1	$\pi(a_1 s_1)$	$\pi(a_2 s_1)$	$\dots$	$\pi(a_m s_1)$	$\leftarrow \pi(a s; \theta)$
s_2	$\pi(a_1 s_2)$	$\pi(a_2 s_2)$	$\dots$	$\pi(a_m s_2)$	
$\vdots$	$\vdots$	$\vdots$		$\vdots$	
s_n	$\pi(a_1 s_n)$	$\pi(a_2 s_n)$	$\dots$	$\pi(a_m s_n)$	

- So far, all methods were based on the action-value function (policies are calculated based on the value of actions $q(s, a)$).
- We need to define the functional $J(\theta)$ to be minimized.

Metric: Average state value

$$\bar{v}_\pi := \sum_{s \in \mathcal{S}} d(s) v_\pi(s) = \mathbb{E}_{S \sim d} [v_\pi(S)],$$

where $d(s)$ is the weight of the state s . It satisfies that $d(s) \geq 0$ para cada $s \in \mathcal{S}$ and $\sum_{s \in \mathcal{S}} d(s) = 1$

Metric: Average (one-step) reward

$$\bar{r}_\pi := \sum_{s \in \mathcal{S}} d_\pi(s) r_\pi(s) = \mathbb{E}_{S \sim d} [v_\pi(S)],$$

where d_π is the stationary distribution and

$$r_\pi(s) := \sum_{a \in \mathcal{A}} \pi(a|s; \theta) r(s, a) = \mathbb{E}_{A \sim \pi(\theta)} [r(S, A) | S = s]$$

Policy gradient methods

Metric	summation	expected value	limit
$\bar{v}_\pi$	$\sum_{s \in \mathcal{S}} d(s) v_\pi(s)$	$\mathbb{E}_{S \sim d} [v_\pi(S)]$	$\lim_{n \rightarrow \infty} \mathbb{E} \left[\sum_{t=0}^n \gamma^t R_{t+1} \right]$
$\bar{r}_\pi$	$\sum_{s \in \mathcal{S}} d_\pi(s) r_\pi(s)$	$\mathbb{E}_{S \sim d_\pi} [r_\pi(S)]$	$\lim_{n \rightarrow \infty} \frac{1}{n} \mathbb{E} \left[\sum_{t=0}^{n-1} R_{t+1} \right]$

- These metrics are functions of π (i.e., functions of θ).
- The functional $J(\theta)$ will be defined in terms of these metrics. Then, we seek θ such that it **maximizes the metrics**.
- To compute the functional gradient, we should consider several cases.

Discounted case ($\gamma < 1$)

After some calculations...

$$\nabla_{\theta} v_{\pi}(s) = \sum_{s' \in \mathcal{S}} \left[(I_n - \gamma P_{\pi})^{-1} \right]_{ss'} \sum_{a \in \mathcal{A}} \nabla_{\theta} \pi(a|s'; \theta) q_{\pi}(s', a),$$

where $[(I_n - \gamma P_{\pi})^{-1}]_{ss'} = \sum_{k=0}^{\infty} \gamma^k [P_{\pi}^k]_{ss'}$.

- $[P_{\pi}]_{ss'}$ is the probability of transitioning from s to s' (in one step).
- $[P_{\pi}^k]_{ss'}$ is the probability of transitioning from s to s' with k steps.
- $\Pr_{\pi}(s'|s) := [(I_n - \gamma P_{\pi})^{-1}]_{ss'}$ is the probability of transitioning from s to s' using any number of steps.

$$\bar{v}_\pi := \mathbb{E}_{S \sim d} [v_\pi(S)] = \sum_{s \in \mathcal{S}} d(s) v_\pi(s) \quad (\text{average state value})$$

Case $d = d_0$ (independent from the policy)

Theorem (Gradient of $\bar{v}_\pi^0$)

In the discounted case where $\gamma \in (0, 1)$ and $d = d_0$ is independent from the policy, the gradient of $\bar{v}_\pi := \bar{v}_\pi^0$ is given by:

$$\nabla_\theta \bar{v}_\pi^0 = \mathbb{E} \left[\nabla_\theta \ln \pi(A|S; \theta) q_\pi(S, A) \right],$$

where $S \sim \rho_\pi$ and $A \sim \pi(S; \theta)$. Here, the state distribution ρ_π is:

$$\rho_\pi(s) = \sum_{s' \in \mathcal{S}} d_0(s') \Pr_\pi(s|s'), \quad s \in \mathcal{S}.$$

$$\bar{v}_\pi := \mathbb{E}_{S \sim d} [v_\pi(S)] = \sum_{s \in \mathcal{S}} d(s) v_\pi(s) \quad (\text{average state value})$$

Case $d = d_\pi$ (stationary distribution)

Theorem (Gradient of $\bar{v}_\pi$)

In the discounted case where $\gamma \in (0, 1)$ y $d = d_\pi$, the gradient of $\bar{v}_\pi$ is given by:

$$\begin{aligned} \nabla_\theta \bar{v}_\pi &\approx \frac{1}{1-\gamma} \sum_{s \in \mathcal{S}} d_\pi(s) \sum_{a \in \mathcal{A}} \nabla_\theta \pi(a|s; \theta) q_\pi(s, a) \\ &= \frac{1}{1-\gamma} \mathbb{E} \left[\nabla_\theta \ln \pi(A|S; \theta) q_\pi(S, A) \right], \end{aligned}$$

where $S \sim d_\pi$ and $A \sim \pi(S; \theta)$.

Obs: In this case, $\bar{r}_\pi = (1-\gamma)\bar{v}_\pi$.

Undiscounted case ($\gamma = 1$)

- We redefine the state and action values:

$$v_{\pi}(s) = \mathbb{E}\left[(R_{t+1} - \bar{r}_{\pi}) + (R_{t+2} - \bar{r}_{\pi}) + \dots | S_t = s\right],$$

$$q_{\pi}(s) = \mathbb{E}\left[(R_{t+1} - \bar{r}_{\pi}) + (R_{t+2} - \bar{r}_{\pi}) + \dots | S_t = s, A_t = a\right].$$

- The Bellman equation is “replaced” by the Poisson equation

$$v_{\pi} = r_{\pi} - \bar{r}_{\pi} \mathbf{1}_n + P_{\pi} v_{\pi}.$$

- The gradient is given by:

$$\nabla_{\theta} \bar{r}_{\pi} = \mathbb{E}\left[\nabla_{\theta} \ln \pi(A|S; \theta) q_{\pi}(S, A)\right],$$

where $S \sim d_{\pi}$ y $A \sim \pi(S; \theta)$.

Summary: (assuming d_π)

	Discounted	Undiscounted
$\bar{v}_\pi$ defined?	Yes, unique	Non-unique
$\bar{r}_\pi$ defined?	Yes, unique	Yes, unique
$\bar{r}_\pi = (1-\gamma)\bar{v}_\pi$	Holds ✓	Breaks down ✗
Policy gradient of $\bar{r}_\pi$	Approximate	Exact
State values satisfy	Bellman equation	Poisson equation

To maximize $J(\theta)$, we will use gradient ascent. In all cases, we can assume that the gradient is given by:

$$\nabla_\theta J(\theta) = \mathbb{E} \left[\nabla_\theta \ln \pi(A|S; \theta) q_\pi(S, A) \right]$$

The policy gradient method REINFORCE can be derived as follows:

$$\theta_{t+1} = \theta_t + \alpha \mathbb{E} \left[\nabla_{\theta} \ln \pi(A | S; \theta_t) q_{\pi}(S, A) \right]$$

↓

$$\theta_{t+1} = \theta_t + \alpha \nabla_{\theta} \ln \pi(a_t | s_t; \theta_t) q_t(s_t, a_t)$$

↓

$$\theta_{t+1} = \theta_t + \alpha \underbrace{\left(\frac{q_t(s_t, a_t)}{\pi(a_t | s_t; \theta_t)} \right)}_{\beta_t} \nabla_{\theta} \ln \pi(a_t | s_t; \theta_t)$$

Machine Learning, 8, 229–256 (1992)

© 1992 Kluwer Academic Publishers, Boston. Manufactured in The Netherlands.

Simple Statistical Gradient-Following Algorithms for Connectionist Reinforcement Learning

RONALD J. WILLIAMS

rjw@corwin.ccs.northeastern.edu

College of Computer Science, 161 CN, Northeastern University, 360 Huntington Ave., Boston, MA 02115

Abstract. This article presents a general class of associative reinforcement learning algorithms for connectionist networks containing stochastic units. These algorithms, called REINFORCE algorithms, are shown to make weight adjustments in a direction that lies along the gradient of expected reinforcement in both immediate-reinforcement tasks and certain limited forms of delayed-reinforcement tasks, and they do this without explicitly computing gradient estimates or even storing information from which such estimates could be computed. Specific examples of such algorithms are presented, some of which bear a close relationship to certain existing algorithms while others are novel but potentially interesting in their own right. Also given are results that show how such algorithms can be naturally integrated with backpropagation. We close with a brief discussion of a number of additional issues surrounding the use of such algorithms, including what is known about their limiting behaviors as well as further considerations that might be used to help develop similar but potentially more powerful reinforcement learning algorithms.

Keywords. Reinforcement learning, connectionist networks, gradient descent, mathematical analysis

Policy gradient methods (REINFORCE)

$$\text{REward Increment} = \text{Nonnegative Factor} \times \text{Offset Reinforcement} \times \text{Characteristic Eligibility}$$

Algorithm REINFORCE

```
1: Input:  $\pi(a|s, \theta)$ ;  $\gamma \in (0, 1)$ ;  $\alpha > 0$ 
2: Initialise: parameter  $\theta$ 
3: loop
4:   Generate an episode  $\{s_0, a_0, r_1, \dots, s_{T-1}, a_{T-1}, r_T\}$  following  $\pi(\theta)$ 
5:   for  $t = 0, 1, \dots, T - 1$  do
6:      $q$ -value update (MC estimation)
7:      $q_t(s_t, a_t) \leftarrow \sum_{k=t+1}^T \gamma^{k-t-1} r_k$ 
8:     Policy update
9:      $\theta \leftarrow \theta + \alpha \left( \frac{q_t(s_t, a_t)}{\pi(a_t|s_t; \theta)} \right) \nabla_{\theta} \pi(a_t|s_t; \theta)$ 
10:   end for
11: end loop
```

Policy gradient methods (REINFORCE)

In <https://github.com/ibrevis/in-deep-week-intro-RL>,

- go to `/cart_pole_example/`
- Go `06_REINFORCE_CartPole.ipynb` and tune the hyperparameters to training the policy.
- Check your results with `06_run_reinforce_cartpole_policy.py`.

